



From Data to Solutions · Weekend 0

NumPy and linear algebra

Carlos Cotrini



A matrix turns amounts into totals; this afternoon a machine learns it

1. A rate table

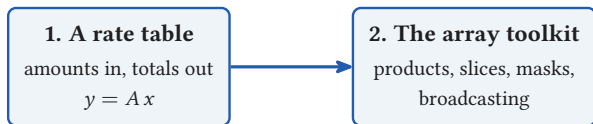
amounts in, totals out

$$y = Ax$$

- we hand-write the rate table A this morning; this afternoon a machine **learns** it



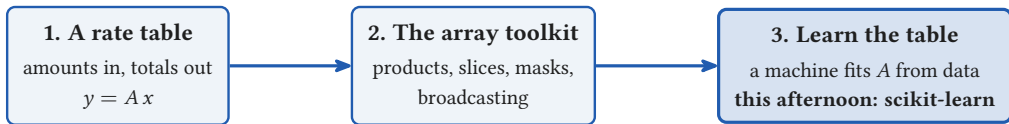
A matrix turns amounts into totals; this afternoon a machine learns it



- we hand-write the rate table A this morning; this afternoon a machine **learns** it
- one running example throughout: a **smoothie recipe** (with coffee and coins along the way)



A matrix turns amounts into totals; this afternoon a machine learns it



- we hand-write the rate table A this morning; this afternoon a machine **learns** it
- one running example throughout: a **smoothie recipe** (with coffee and coins along the way)



Section 1

Matrices as transformations

How does one table of rates turn a list of amounts into a list of totals?



A rate card turns the scoops you used into a nutrition label

scoops you used

3
2
2
1

oats, p. butter,
whey, banana

A rate card turns the scoops you used into a nutrition label

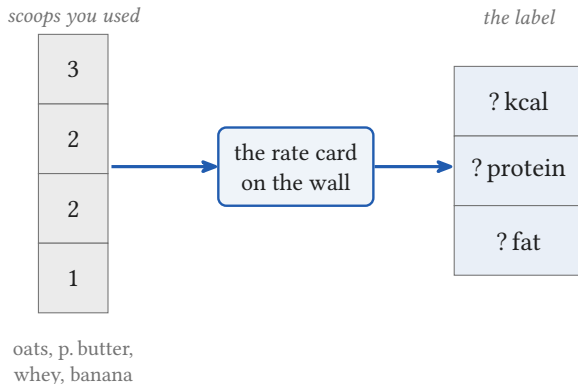
scoops you used

3
2
2
1

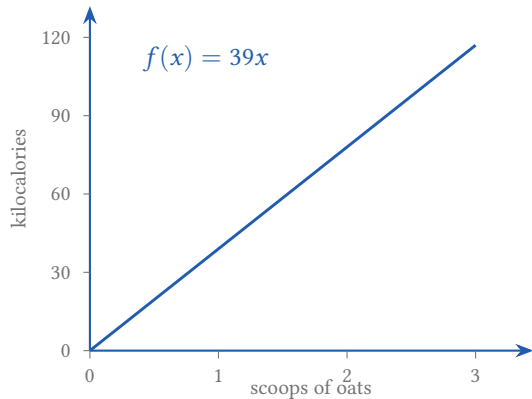
the rate card
on the wall

oats, p. butter,
whey, banana

A rate card turns the scoops you used into a nutrition label

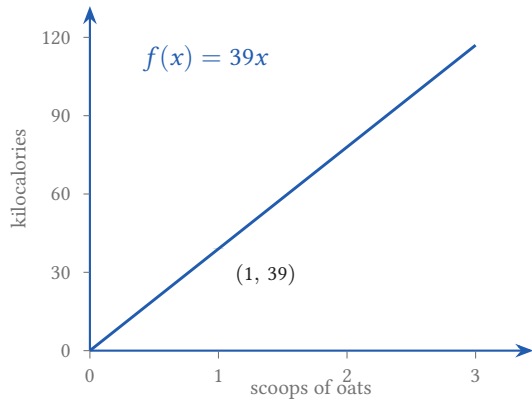


A linear function scales its one input by a constant rate



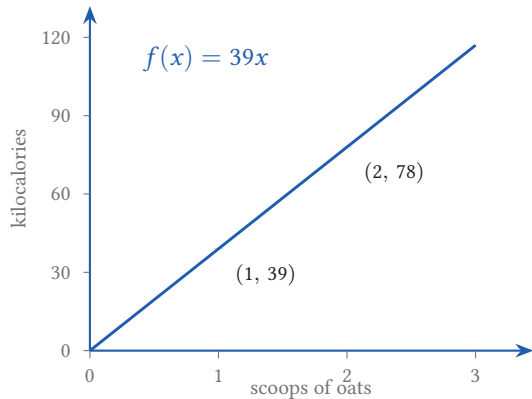
- one input, one output:
 $f(x) = ax$ with $a = 39$

A linear function scales its one input by a constant rate



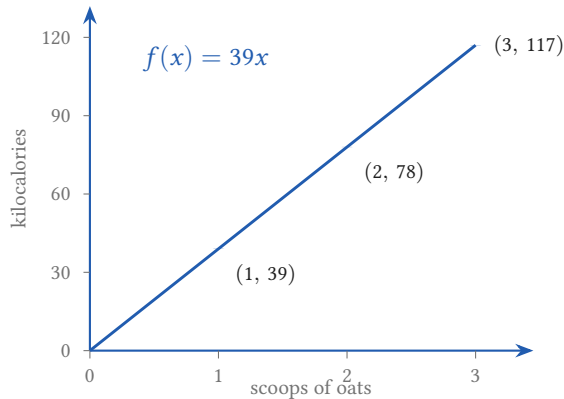
- one input, one output:
 $f(x) = ax$ with $a = 39$
- the output is a constant **multiple** of the input

A linear function scales its one input by a constant rate



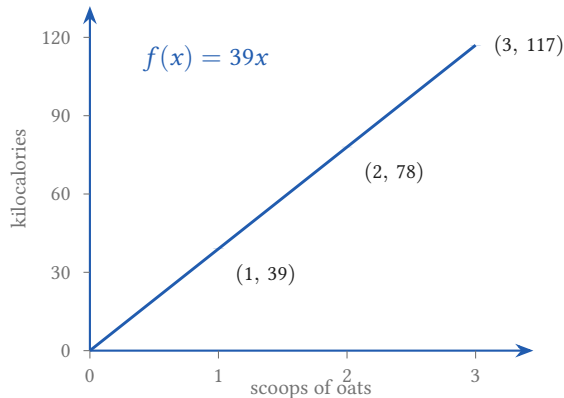
- one input, one output:
 $f(x) = ax$ with $a = 39$
- the output is a constant **multiple** of the input
- double the scoops, double the kilocalories

A linear function scales its one input by a constant rate



- one input, one output:
 $f(x) = ax$ with $a = 39$
- the output is a constant **multiple** of the input
- double the scoops, double the kilocalories
- the graph is a straight **line** through the origin

A linear function scales its one input by a constant rate



- one input, one output:
 $f(x) = ax$ with $a = 39$
- the output is a constant **multiple** of the input
- double the scoops, double the kilocalories
- the graph is a straight **line through the origin**
- the rate $a = 39$ is its **slope**

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) =$$

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3$$

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3 + 59 \cdot 2$$

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2$$

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1$$

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339 \text{ kcal}$$

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339 \text{ kcal}$$

- four inputs, one output

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	kcal rate per scoop
x	3	2	2	1	scoops you used

$$f(x) = 39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339 \text{ kcal}$$

- four inputs, one output
- the output is a **weighted sum**: each input scaled by its rate, then added

Several inputs combine as a weighted sum

	oats	p.butter	whey	banana	
a	39	59	37	30	<i>kcal rate per scoop</i>
	▼	▼	▼	▼	
x	3	2	2	1	<i>scoops you used</i>

$$f(x) = 39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339 \text{ kcal}$$

- four inputs, one output
- the output is a **weighted sum**: each input scaled by its rate, then added
- that weighted sum is exactly the dot product $a \cdot x$, a **linear** function

Stacking three rate rows builds the product $y = A x$

					scoops x	
					3	oats
					2	p.butter
					2	whey
					1	banana
A	kcal	39	59	37	30	339
	protein					
	fat					

- each row of A is one linear function of the scoops

Stacking three rate rows builds the product $y = Ax$

					scoops x	
					3	oats
					2	p.butter
					2	whely
					1	banana
A	kcal	39	59	37	30	339
	protein	1.7	2.5	8.0	0.3	
	fat					

- each **row** of A is one linear function of the scoops
- kcal, protein, fat: three rates, stacked as rows

Stacking three rate rows builds the product $y = Ax$

					scoops x	
					3	oats
					2	p.butter
					2	whey
					1	banana
A	oats	p.butter	whey	banana		
	kcal	39	59	37	30	
	protein	1.7	2.5	8.0	0.3	
	fat	0.7	5.0	0.7	0.0	
					339	
					26.4	
					13.5	
					totals y	

- each row of A is one linear function of the scoops
- kcal, protein, fat: three rates, stacked as rows
- all at once is the matrix-vector product $y = Ax$

Stacking three rate rows builds the product $y = Ax$

					scoops x	
					3	oats
					2	p.butter
					2	whey
					1	banana
A	oats	p.butter	whey	banana		
	kcal	39	59	37	30	
	protein	1.7	2.5	8.0	0.3	
	fat	0.7	5.0	0.7	0.0	
					339	
					26.4	
					13.5	
					totals y	

- each row of A is one linear function of the scoops
- kcal, protein, fat: three rates, stacked as rows
- all at once is the matrix-vector product $y = Ax$
- this product runs through the whole lecture

Each row of A is a nutrient, each column a food

	oats	p.butter	whey	banana	
kcal	39	59	37	30	shape (3, 4)
protein	1.7	2.5	8.0	0.3	3 nutrients (rows)
fat	0.7	5.0	0.7	0.0	4 foods (columns)

- each **row** is one nutrient's rate per food (row 0: kcal per scoop)

Each row of A is a nutrient, each column a food

	oats	p.butter	whey	banana	
kcal	39	59	37	30	shape (3, 4)
protein	1.7	2.5	8.0	0.3	3 nutrients (rows)
fat	0.7	5.0	0.7	0.0	4 foods (columns)

- each **row** is one nutrient's rate per food (row 0: kcal per scoop)
- each **column** is one food's full nutrition profile

Each row of A is a nutrient, each column a food

	oats	p.butter	whey	banana	
kcal	39	59	37	30	shape (3, 4)
protein	1.7	2.5	8.0	0.3	3 nutrients (rows)
fat	0.7	5.0	0.7	0.0	4 foods (columns)

- each **row** is one nutrient's rate per food (row 0: kcal per scoop)
- each **column** is one food's full nutrition profile
- two more matrices follow: **coffee** (3, 2) and **coins** (2, 3)



Section 2

Building arrays

How do we lay these numbers out so that the products line up?

2

One smoothie per column lets A @ X line up

X =

	smoothie 0	smoothie 1	smoothie 2
oats	3	1	2
p.butter	2	0	2
whey	2	3	0
banana	1	2	0

- one smoothie per **column**, one food per **row**: a (4, 3) grid

One smoothie per column lets $A @ X$ line up

$X =$

	smoothie 0	smoothie 1	smoothie 2
oats	3	1	2
p.butter	2	0	2
whey	2	3	0
banana	1	2	0

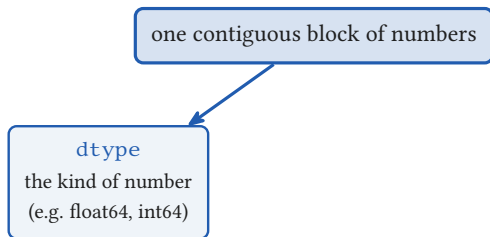
- one smoothie per **column**, one food per **row**: a (4, 3) grid
- this lines up with A : its 4 columns (foods) meet X 's 4 rows (foods), so $A @ X$ works

An array is one block of numbers plus a shape

one contiguous block of numbers

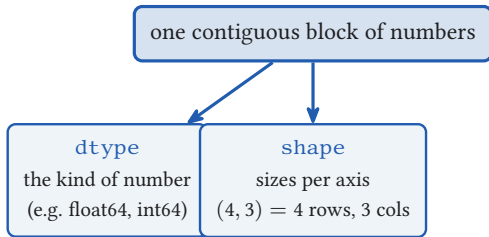
- a 1-D array has shape `(n, )` (the trailing comma marks one axis)

An array is one block of numbers plus a shape



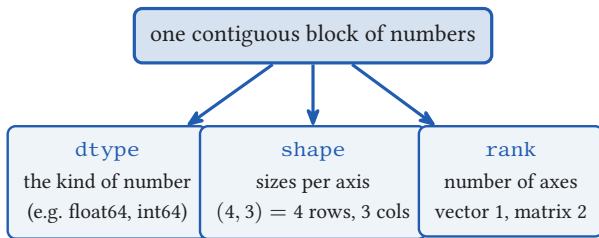
- a 1-D array has shape `(n, )` (the trailing comma marks one axis)
- maths counts from 1, NumPy counts from 0: `x[0, 0]` is the first entry

An array is one block of numbers plus a shape



- a 1-D array has shape `(n, )` (the trailing comma marks one axis)
- maths counts from 1, NumPy counts from 0: `x[0, 0]` is the first entry

An array is one block of numbers plus a shape



- a 1-D array has shape `(n, )` (the trailing comma marks one axis)
- maths counts from 1, NumPy counts from 0: `x[0, 0]` is the first entry

A list of lists becomes a grid you can shape

X =

	smoothie 0	smoothie 1	smoothie 2
oats	3.	1.	2.
p.butter	2.	0.	2.
whey	2.	3.	0.
banana	1.	2.	0.

- `np.array([[3., 1., 2.], [2., 0., 2.], ...])`: a list of lists becomes the grid

A list of lists becomes a grid you can shape

X =

	smoothie 0	smoothie 1	smoothie 2
oats	3.	1.	2.
p.butter	2.	0.	2.
whey	2.	3.	0.
banana	1.	2.	0.

X.shape is (4,3)

- `np.array([[3., 1., 2.], [2., 0., 2.], ...])`: a list of lists becomes the grid
- no typing: `np.zeros((4,3))`, `np.ones(...)`, `np.arange(3)` is `[0, 1, 2]`

A list of lists becomes a grid you can shape

X =

	smoothie 0	smoothie 1	smoothie 2
oats	3.	1.	2.
p.butter	2.	0.	2.
whey	2.	3.	0.
banana	1.	2.	0.

X.shape is (4, 3)
X.dtype is float64

- `np.array([[3., 1., 2.], [2., 0., 2.], ...])`: a list of lists becomes the grid
- no typing: `np.zeros((4, 3))`, `np.ones(...)`, `np.arange(3)` is `[0, 1, 2]`
- the gotcha: `3.` gives float64 but bare `3` gives int64, which truncates on divide

A few constructors build any array you need

```
np.array([[1,2],[3,4]])
```

a 2×2 from a list of lists

A few constructors build any array you need

```
np.array([[1,2],[3,4]])
```

a 2×2 from a list of lists

```
np.zeros((3,4))
```

a 3×4 filled with 0

A few constructors build any array you need

`np.array([[1,2],[3,4]])`

a 2×2 from a list of lists

`np.zeros((3,4))`

a 3×4 filled with 0

`np.ones((2,3))`

a 2×3 filled with 1

A few constructors build any array you need

`np.array([[1,2],[3,4]])`

a 2×2 from a list of lists

`np.zeros((3,4))`

a 3×4 filled with 0

`np.ones((2,3))`

a 2×3 filled with 1

`np.full((2,2), 7)`

a 2×2 filled with 7

A few constructors build any array you need

`np.array([[1,2],[3,4]])`

a 2×2 from a list of lists

`np.zeros((3,4))`

a 3×4 filled with 0

`np.ones((2,3))`

a 2×3 filled with 1

`np.full((2,2), 7)`

a 2×2 filled with 7

`np.arange(6).reshape(2,3)`

0, 1, ..., 5 folded into a 2×3

A few constructors build any array you need

`np.array([[1,2],[3,4]])`

a 2×2 from a list of lists

`np.zeros((3,4))`

a 3×4 filled with 0

`np.ones((2,3))`

a 2×3 filled with 1

`np.full((2,2), 7)`

a 2×2 filled with 7

`np.arange(6).reshape(2,3)`

0, 1, ..., 5 folded into a 2×3

`np.eye(3)`

the 3×3 identity matrix

A few constructors build any array you need

`np.array([[1,2],[3,4]])`

a 2×2 from a list of lists

`np.zeros((3,4))`

a 3×4 filled with 0

`np.ones((2,3))`

a 2×3 filled with 1

`np.full((2,2), 7)`

a 2×2 filled with 7

`np.arange(6).reshape(2,3)`

0, 1, ..., 5 folded into a 2×3

`np.eye(3)`

the 3×3 identity matrix

- pass the shape as a **tuple**: `np.zeros((3,4))`, not `np.zeros(3,4)`

A few constructors build any array you need

`np.array([[1,2],[3,4]])`

a 2×2 from a list of lists

`np.zeros((3,4))`

a 3×4 filled with 0

`np.ones((2,3))`

a 2×3 filled with 1

`np.full((2,2), 7)`

a 2×2 filled with 7

`np.arange(6).reshape(2,3)`

0, 1, ..., 5 folded into a 2×3

`np.eye(3)`

the 3×3 identity matrix

- pass the shape as a **tuple**: `np.zeros((3,4))`, not `np.zeros(3,4)`
- **reshape** keeps the data; the new sizes must multiply to the same count



Section 3

The dot product and similarity

When is one list of numbers similar to another?

3

■ The dot product and similarity

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	<i>calorie rates</i>
x	3	2	2	1	<i>scoops</i>

■ The dot product and similarity

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	<i>calorie rates</i>
	▼	▼	▼	▼	
x	3	2	2	1	<i>scoops</i>

■ The dot product and similarity

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	calorie rates
x	3	2	2	1	scoops

$39 \cdot 3$
oats

■ The dot product and similarity

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	calorie rates
x	3	2	2	1	scoops

$$39 \cdot 3 + \underbrace{59 \cdot 2}_{\text{p. butter}}$$

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	calorie rates
	▼	▼	▼	▼	
x	3	2	2	1	scoops

$$39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1$$

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	calorie rates
x	3	2	2	1	scoops

$$39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339 \text{ kcal}$$

One row dotted with the scoops gives one total

	oats	p.butter	whey	banana	
u	39	59	37	30	calorie rates
x	3	2	2	1	scoops

$$39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339 \text{ kcal}$$

one row dotted with the scoops gives one total

A vector's norm is its length

$$\|x\| = \sqrt{x_1^2 + \cdots + x_n^2}$$

- the Euclidean norm is the **length** of the vector

A vector's norm is its length

$$\|x\| = \underbrace{\sqrt{x_1^2 + \cdots + x_n^2}}_{\text{the length}}$$

- the Euclidean norm is the **length** of the vector
- **normalizing** divides a vector by its own norm: length becomes 1, proportions kept

Dividing a vector by its norm sets its length to one

$$\mathbf{x} = \begin{bmatrix} 3 & 0 & 4 \end{bmatrix}$$

- here $\sqrt{3^2 + 0^2 + 4^2} = 5$; use `np.linalg.norm(X, axis=1)` for one length per batch row

Dividing a vector by its norm sets its length to one

x

3	0	4
---	---	---

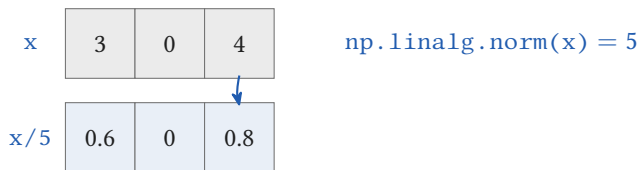
`np.linalg.norm(x) = 5`

- here $\sqrt{3^2 + 0^2 + 4^2} = 5$; use `np.linalg.norm(X, axis=1)` for one length per batch row
- dividing gives $[0.6, 0, 0.8]$, of length 1 (proportions kept)

Dividing a vector by its norm sets its length to one

x	3	0	4
x/5	0.6	0	0.8

$\text{np.linalg.norm}(x) = 5$



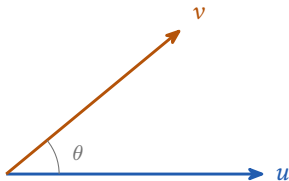
- here $\sqrt{3^2 + 0^2 + 4^2} = 5$; use `np.linalg.norm(X, axis=1)` for one length per batch row
- dividing gives $[0.6, 0, 0.8]$, of length 1 (proportions kept)
- guard it: an int-dtype vector truncates, and a zero vector divides by zero

Dividing a vector by its norm sets its length to one

x	3	0	4	<code>np.linalg.norm(x) = 5</code>
x/5	0.6	0	0.8	length = 1

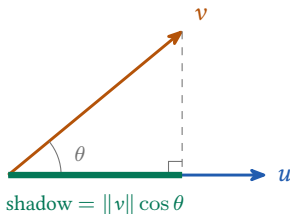
- here $\sqrt{3^2 + 0^2 + 4^2} = 5$; use `np.linalg.norm(X, axis=1)` for one length per batch row
- dividing gives $[0.6, 0, 0.8]$, of length 1 (proportions kept)
- guard it: an int-dtype vector truncates, and a zero vector divides by zero

The dot product is a length times a shadow



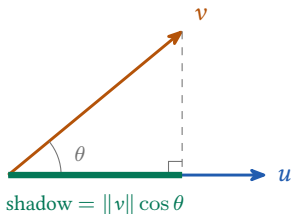
- for **any** two vectors u and v , drop v onto the line of u

The dot product is a length times a shadow



- for **any** two vectors u and v , drop v onto the line of u
- the foot marks v 's **shadow**, of length $\|v\| \cos \theta$

The dot product is a length times a shadow



$$u \cdot v = \|u\| \|v\| \cos \theta = \|u\| \times (\text{the shadow of } v \text{ on } u)$$

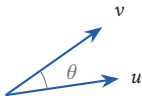
- for **any** two vectors u and v , drop v onto the line of u
- the foot marks v 's **shadow**, of length $\|v\| \cos \theta$
- “multiply, then add” is exactly that shadow times $\|u\|$

The dot product measures how two vectors align

$$u \cdot v = \|u\| \|v\| \cos \theta$$

The dot product measures how two vectors align

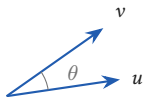
$$u \cdot v = \|u\| \|v\| \cos \theta$$



$u \cdot v > 0$
aligned

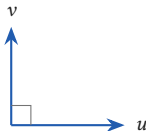
- the dot product grows as two vectors line up

The dot product measures how two vectors align



$u \cdot v > 0$
aligned

$$u \cdot v = \|u\| \|v\| \cos \theta$$

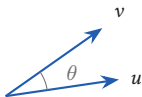


$u \cdot v = 0$
perpendicular

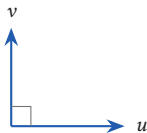
- the dot product grows as two vectors line up
- it is 0 exactly when they are perpendicular ($\cos 90^\circ = 0$)

The dot product measures how two vectors align

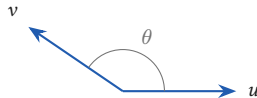
$$u \cdot v = \|u\| \|v\| \cos \theta$$



$u \cdot v > 0$
aligned



$u \cdot v = 0$
perpendicular

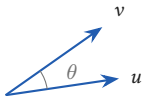


$u \cdot v < 0$
opposed

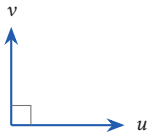
- the dot product grows as two vectors line up
- it is 0 exactly when they are perpendicular ($\cos 90^\circ = 0$)
- it turns negative when they point opposite ways ($\cos \theta < 0$)

The dot product measures how two vectors align

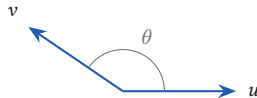
$$u \cdot v = \|u\| \|v\| \cos \theta$$



$u \cdot v > 0$
aligned



$u \cdot v = 0$
perpendicular



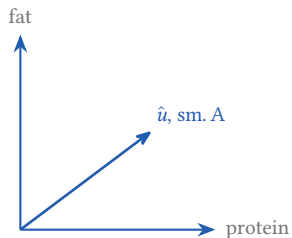
$u \cdot v < 0$
opposed

- the dot product grows as two vectors line up
- it is 0 exactly when they are perpendicular ($\cos 90^\circ = 0$)
- it turns negative when they point opposite ways ($\cos \theta < 0$)

aligned is positive, perpendicular is zero, opposed is negative

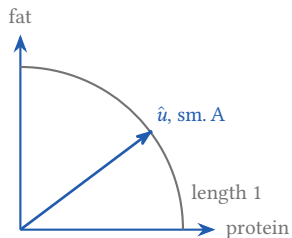
Normalize, then dot: the cosine similarity score

$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$



Normalize, then dot: the cosine similarity score

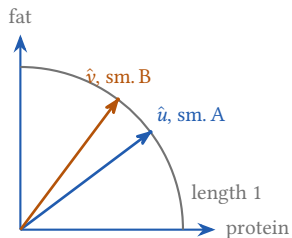
$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$



Normalize, then dot: the cosine similarity score

$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$

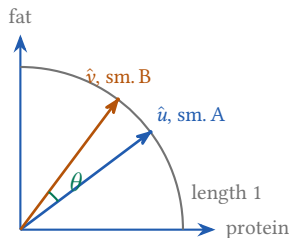
$$\text{sm. B: } \hat{v} = (0.6, 0.8)$$



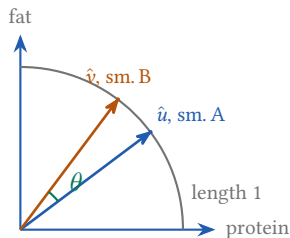
Normalize, then dot: the cosine similarity score

$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$

$$\text{sm. B: } \hat{v} = (0.6, 0.8)$$



Normalize, then dot: the cosine similarity score



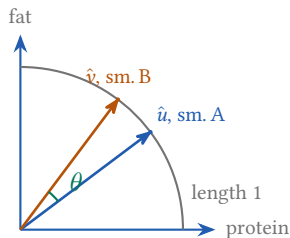
$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$

$$\text{sm. B: } \hat{v} = (0.6, 0.8)$$

$$\hat{u} \cdot \hat{v} = \cos \theta = 0.96$$

96% similar

Normalize, then dot: the cosine similarity score



$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$

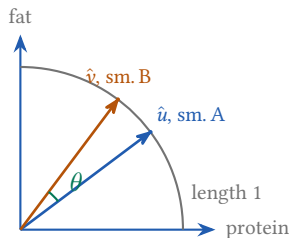
$$\text{sm. B: } \hat{v} = (0.6, 0.8)$$

$$\hat{u} \cdot \hat{v} = \cos \theta = 0.96$$

96% similar

- normalize, then dot: a pure score, cosine similarity

Normalize, then dot: the cosine similarity score



$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$

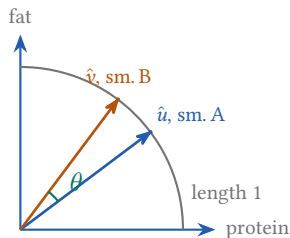
$$\text{sm. B: } \hat{v} = (0.6, 0.8)$$

$$\hat{u} \cdot \hat{v} = \cos \theta = 0.96$$

96% similar

- normalize, then dot: a pure score, cosine similarity
- +1 same direction, 0 unrelated, -1 opposite

Normalize, then dot: the cosine similarity score



$$\text{sm. A: } \hat{u} = (0.8, 0.6)$$

$$\text{sm. B: } \hat{v} = (0.6, 0.8)$$

$$\hat{u} \cdot \hat{v} = \cos \theta = 0.96$$

96% similar

- normalize, then dot: a pure score, **cosine similarity**
- +1 same direction, 0 unrelated, -1 opposite
- A and B share a protein-to-fat **mix**, regardless of serving size



Section 4

The matrix-vector product

What does multiplying a whole table by one vector compute?



A matrix-vector product is one dot product per row

$$\langle u, v \rangle = \sum_i u_i v_i$$

A matrix-vector product is one dot product per row

$$\langle u, v \rangle = \sum_i u_i v_i$$

- two equal-length lists in, one number out (the lengths must match)

A matrix-vector product is one dot product per row

$$\langle u, v \rangle = \sum_i u_i v_i$$

- two equal-length lists in, one number out (the lengths must match)
- “multiply matching entries, then add”: `u @ x`

■ The matrix-vector product

A matrix-vector product is one dot product per row

$$y_i = \sum_j A[i, j] x_j \quad (y = A x)$$



row



column



result

■ The matrix-vector product

A matrix-vector product is one dot product per row

$$y_i = \sum_j A[i, j] x_j \quad (y = A x)$$



row



column



result

- shape contract: (m, n) @ $(n,)$ is $(m,)$; the inner n 's meet and **vanish**

■ The matrix-vector product

A matrix-vector product is one dot product per row

$$y_i = \sum_j A[i, j] x_j \quad (y = A x)$$



row



column



result

- shape contract: (m, n) @ $(n,)$ is $(m,)$; the inner n 's meet and **vanish**
- **row** picture: y_i is row i dotted with x (what is each total?)

■ The matrix-vector product

A matrix-vector product is one dot product per row

$$y_i = \sum_j A[i, j] x_j \quad (y = A x)$$



row



column



result

- shape contract: (m, n) @ $(n,)$ is $(m,)$; the inner n 's meet and **vanish**
- **row** picture: y_i is row i dotted with x (what is each total?)
- **column** picture: $y = \sum_j x_j A[:, j]$, a weighted blend (how much did each food add?)

■ The matrix-vector product

A matrix-vector product is one dot product per row

$$y_i = \sum_j A[i, j] x_j \quad (y = A x)$$



row



column



result

- shape contract: $(m, n) @ (n,)$ is $(m,)$; the inner n 's meet and **vanish**
- **row** picture: y_i is row i dotted with x (what is each total?)
- **column** picture: $y = \sum_j x_j A[:, j]$, a weighted blend (how much did each food add?)
- both give the same $[339, 26.4, 13.5]$

■ The matrix-vector product

Row picture: each total is one row of A dotted with x

		scoops x			
		oats	p.butter	whey	banana
A	kcal	39	59	37	30
	protein	1.7	2.5	8.0	0.3
	fat	0.7	5.0	0.7	0.0

scoops x	
3	oats
2	p.butter
2	whey
1	banana

- y_i is row i of A , dotted with x

■ The matrix-vector product

Row picture: each total is one row of A dotted with x

		scoops x			
		oats	p.butter	whey	banana
A	kcal	39	59	37	30
	protein	1.7	2.5	8.0	0.3
	fat	0.7	5.0	0.7	0.0
		totals y			
		339			

■ y_i is row i of A , dotted with x

■ row 0:

$$39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339$$

■ The matrix-vector product

Row picture: each total is one row of A dotted with x

		scoops x			
		oats	p.butter	whey	banana
A	kcal	39	59	37	30
	protein	1.7	2.5	8.0	0.3
	fat	0.7	5.0	0.7	0.0
		totals y			
		339	26.4	13.5	

- y_i is row i of A , dotted with x
- row 0:
 $39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339$
- rows 1, 2 give protein 26.4, fat 13.5

■ The matrix-vector product

Row picture: each total is one row of A dotted with x

		scoops x			
		oats	p.butter	whey	banana
A	kcal	39	59	37	30
	protein	1.7	2.5	8.0	0.3
	fat	0.7	5.0	0.7	0.0
		totals y			
		339	26.4	13.5	

- y_i is row i of A , dotted with x
- row 0:
 $39 \cdot 3 + 59 \cdot 2 + 37 \cdot 2 + 30 \cdot 1 = 339$
- rows 1, 2 give protein 26.4, fat 13.5
- x matches A 's column count:
 $y = A @ x$

Column picture: each column is where a food lands

$$y = \begin{matrix} & \text{oats} \\ & 3\times \\ \text{kcal} & 39 \\ \text{protein} & 1.7 \\ \text{fat} & 0.7 \end{matrix} +$$

Column picture: each column is where a food lands

$$y = \begin{matrix} & \text{oats} & & \text{p.butter} \\ & 3\times & & 2\times \\ \text{kcal} & \begin{bmatrix} 39 \\ 1.7 \\ 0.7 \end{bmatrix} & + & \begin{bmatrix} 59 \\ 2.5 \\ 5.0 \end{bmatrix} & + \end{matrix}$$

Column picture: each column is where a food lands

$$y = \begin{matrix} & \text{oats} & & \text{p.butter} & & \text{whey} \\ & 3\times & & 2\times & & 2\times \\ \text{kcal} & \begin{bmatrix} 39 \\ 1.7 \\ 0.7 \end{bmatrix} & + & \begin{bmatrix} 59 \\ 2.5 \\ 5.0 \end{bmatrix} & + & \begin{bmatrix} 37 \\ 8.0 \\ 0.7 \end{bmatrix} & + \end{matrix}$$

Column picture: each column is where a food lands

$$y = \begin{matrix} & \text{oats} & \text{p.butter} & \text{whey} & \text{banana} \\ & 3\times & 2\times & 2\times & 1\times \\ \text{kcal} & \begin{bmatrix} 39 \\ 1.7 \\ 0.7 \end{bmatrix} & + \begin{bmatrix} 59 \\ 2.5 \\ 5.0 \end{bmatrix} & + \begin{bmatrix} 37 \\ 8.0 \\ 0.7 \end{bmatrix} & + \begin{bmatrix} 30 \\ 0.3 \\ 0.0 \end{bmatrix} & = \end{matrix}$$

Column picture: each column is where a food lands

		oats		p.butter		whey		banana		totals y
		3×		2×		2×		1×		
y =	kcal	39		59		37		30		339
	protein	1.7	+	2.5	+	8.0	+	0.3	=	26.4
	fat	0.7		5.0		0.7		0.0		13.5

Column picture: each column is where a food lands

		oats		p.butter		whey		banana		totals y
		3×		2×		2×		1×		
y =	kcal	39		59		37		30		339
	protein	1.7	+	2.5	+	8.0	+	0.3	=	26.4
	fat	0.7		5.0		0.7		0.0		13.5

- each **column** of A is where one food **lands** in (kcal, protein, fat) space

Column picture: each column is where a food lands

		oats		p.butter		whey		banana		totals y
		3×		2×		2×		1×		
y =	kcal	39		59		37		30		339
	protein	1.7	+	2.5	+	8.0	+	0.3	=	26.4
	fat	0.7		5.0		0.7		0.0		13.5

- each **column** of A is where one food **lands** in (kcal, protein, fat) space
- scale that column by its scoops x_j , then add the four landings

Column picture: each column is where a food lands

	oats		p.butter		whey		banana		totals y
	$3\times$		$2\times$		$2\times$		$1\times$		
$y =$	kcal	39	+	59	+	37	+	30	339
	protein	1.7		2.5		8.0		0.3	26.4
	fat	0.7		5.0		0.7		0.0	13.5

- each **column** of A is where one food **lands** in (kcal, protein, fat) space
- scale that column by its scoops x_j , then add the four landings
- that blend is exactly what “ A **transforms** the scoops into”, the same $[339, 26.4, 13.5]$



Section 5

The matrix-matrix product

How do we transform many inputs at once?

5

Coffee: a bean rate table A and the blend recipes B

		House	Dark
A	cost	8	15
	caffeine	12	20
	weight	1	1

per kg of bean

- A : each bean's cost, caffeine, weight (House and Dark), per kg

Coffee: a bean rate table A and the blend recipes B

		House	Dark
A	cost	8	15
	caffeine	12	20
	weight	1	1

per kg of bean

		blend A	blend B	blend C
B	House	2	1	0
	Dark	3	1	4

kg of bean per blend

- A : each bean's cost, caffeine, weight (House and Dark), per kg
- B : each blend's recipe, how many kg of each bean it uses

Coffee: a bean rate table A and the blend recipes B

A

	House	Dark
cost	8	15
caffeine	12	20
weight	1	1

per kg of bean

B

	blend A	blend B	blend C
House	2	1	0
Dark	3	1	4

kg of bean per blend

- A : each bean's cost, caffeine, weight (House and Dark), per kg
- B : each blend's recipe, how many kg of each bean it uses
- we want cost, caffeine, weight **per blend**: that is $A @ B$

The matrix-matrix product

$A @ B$ prices every coffee blend in one product

B			
blend A	blend B	blend C	
2	1	0	House
3	1	4	Dark

		House	Dark
A	cost	8	15
	caffeine	12	20
	weight	1	1

- Y_{ij} = row i of A dotted with column j of B

The matrix-matrix product

A @ B prices every coffee blend in one product

		<i>B</i>			
		blend A	blend B	blend C	
<i>A</i>	cost	8	15		
	caffeine	12	20		
	weight	1	1		

<i>B</i>			
blend A	blend B	blend C	
2	1	0	House
3	1	4	Dark

	61	
--	----	--

$$Y = AB$$

- Y_{ij} = row i of A dotted with column j of B
- cost row · blend A column:
 $8 \cdot 2 + 15 \cdot 3 = 61$

■ The matrix-matrix product

A @ B prices every coffee blend in one product

		<i>B</i>			
		blend A	blend B	blend C	
<i>A</i>	cost	8	15		
	caffeine	12	20		
	weight	1	1		

		<i>B</i>			
		blend A	blend B	blend C	
	House	2	1	0	House
	Dark	3	1	4	Dark

$$Y = AB$$

- Y_{ij} = row i of A dotted with column j of B
- cost row · blend A column:
 $8 \cdot 2 + 15 \cdot 3 = 61$
- A 's 2 columns (beans) meet B 's 2 rows; the inner 2 vanishes

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

$$C[i,j] = \sum_p A[i,p] B[p,j]$$



row



column



result

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

$$C[i, j] = \sum_p A[i, p] B[p, j]$$



row



column



result

- column j of C is A applied to column j of B

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

$$C[i, j] = \sum_p A[i, p] B[p, j]$$



row



column



result

- column j of C is A applied to column j of B
- shape contract: $(m, k) @ (k, n)$ is (m, n) ; inner k meets and vanishes

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

$$C[i, j] = \sum_p A[i, p] B[p, j]$$



row



column



result

- **column** j of C is A applied to **column** j of B
- shape contract: $(m, k) @ (k, n)$ is (m, n) ; inner k meets and **vanishes**
- the matrix-vector product is the one-column special case

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

a batch X stacks inputs as **columns**: transform with $A @ X$

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

a batch X stacks inputs as **columns**: transform with $A @ X$

- stack N inputs as the **columns** of X , shape (n, N)

■ The matrix-matrix product

One product $C = AB$ transforms every column at once

a batch X stacks inputs as **columns**: transform with $A @ X$

- stack N inputs as the **columns** of X , shape (n, N)
- A is (m, n) : its n columns meet X 's n rows, the inner n **vanishes**

One product $C = AB$ transforms every column at once

a batch X stacks inputs as **columns**: transform with $A @ X$

- stack N inputs as the **columns** of X , shape (n, N)
- A is (m, n) : its n columns meet X 's n rows, the inner n **vanishes**
- result $A @ X$ is (m, N) : one output column per input column

One product $C = AB$ transforms every column at once

a batch X stacks inputs as **columns**: transform with $A @ X$

- stack N inputs as the **columns** of X , shape (n, N)
- A is (m, n) : its n columns meet X 's n rows, the inner n **vanishes**
- result $A @ X$ is (m, N) : one output column per input column
- lay the batch out to match A , and no transpose is needed

The matrix-matrix product

$A @ X$ labels all three smoothies in one product

		X			
		sm.0	sm.1	sm.2	
A	kcal	39	59	37	30
	prot	1.7	2.5	8.0	0.3
	fat	0.7	5.0	0.7	0.0

		X			
		sm.0	sm.1	sm.2	
	oats	3	1	2	
	p.b.	2	0	2	
	whey	2	3	0	
	ban.	1	2	0	

- Y_{ij} = row i of A dotted with column j of X

The matrix-matrix product

A @ X labels all three smoothies in one product

		X				
		sm.0	sm.1	sm.2		
A	kcal	3	1	2		oats
	prot	2	0	2		p.b.
		2	3	0		whey
	fat	1	2	0		ban.

		oats	p.b.	whey	ban.
A	kcal	39	59	37	30
	prot	1.7	2.5	8.0	0.3
	fat	0.7	5.0	0.7	0.0

339

$$Y = AX$$

- Y_{ij} = row i of A dotted with column j of X
- $\text{kcal} \cdot \text{sm.0} = 339$ (top-left of Y)

■ The matrix-matrix product

A @ X labels all three smoothies in one product

		X					
		sm.0	sm.1	sm.2			
A	kcal	3	1	2			oats
	prot	2	0	2			p.b.
		2	3	0			whey
	fat	1	2	0			ban.
		oats	p.b.	whey	ban.		
Y = A X	kcal	339	210	196			kcal
	prot	26.4	26.3	8.4			prot
		13.5	2.8	11.4			fat

- Y_{ij} = row i of A dotted with column j of X
- $\text{kcal} \cdot \text{sm.0} = 339$ (top-left of Y)
- A 's 4 columns meet X 's 4 rows; the inner 4 vanishes, $A @ X$ is $(3, 3)$

Transposing swaps rows and columns to line up a product

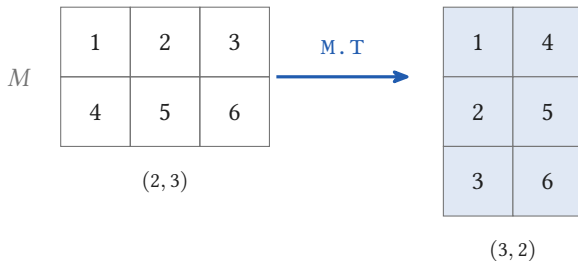
M

1	2	3
4	5	6

$(2, 3)$

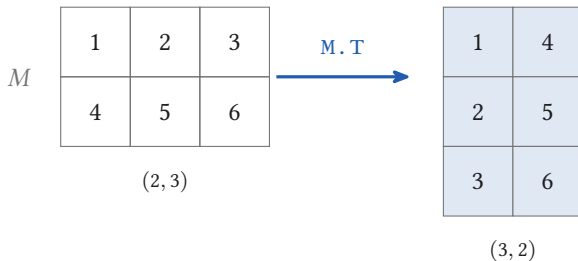
- $M.T$ swaps rows and columns: (m, n) becomes (n, m) , entry $[i, j]$ becomes $[j, i]$

Transposing swaps rows and columns to line up a product



- $M.T$ swaps rows and columns: (m, n) becomes (n, m) , entry $[i, j]$ becomes $[j, i]$
- use it to reorient an array so its axes line up for a product

Transposing swaps rows and columns to line up a product



- $M.T$ swaps rows and columns: (m, n) becomes (n, m) , entry $[i, j]$ becomes $[j, i]$
- use it to reorient an array so its axes line up for a product
- we stored X as $(\text{foods} \times \text{smoothies})$ on purpose, so $A @ X$ needs no $.T$



Section 6

Matrix product versus elementwise product

Why is one keystroke, $$ or $@$, a completely different operation?*

6

Coins: multiplying gives three numbers, but you still must add

	1-franc	2-franc	5-franc	
$[1, 2, 5]$	1	2	5	<i>value per coin</i>
$[10, 5, 4]$	10	5	4	<i>counts</i>

- multiply rates by counts: $[10, 10, 20]$, three numbers, one per coin kind

Coins: multiplying gives three numbers, but you still must add

	1-franc	2-franc	5-franc	
$[1, 2, 5]$	1	2	5	<i>value per coin</i>
$[10, 5, 4]$	10	5	4	<i>counts</i>

- multiply rates by counts: $[10, 10, 20]$, three numbers, one per coin kind
- you still must ADD them to reach 40 CHF

Coins: multiplying gives three numbers, but you still must add

	1-franc	2-franc	5-franc	
$[1, 2, 5]$	1	2	5	<i>value per coin</i>
$[10, 5, 4]$	10	5	4	<i>counts</i>

- multiply rates by counts: $[10, 10, 20]$, three numbers, one per coin kind
- you still must ADD them to reach 40 CHF
- the gap is one keystroke, $*$ versus $@$; the wrong one never warns you

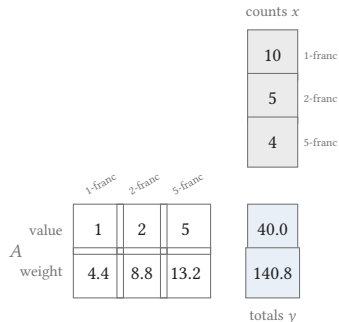
Matrix product versus elementwise product

@ multiplies and sums; * multiplies and stops

		counts x		
		10	5	4
		1-franc	2-franc	5-franc
A	value	1	2	5
	weight	4.4	8.8	13.2
		totals y		

Matrix product versus elementwise product

@ multiplies and sums; * multiplies and stops



3 counts in, 2 totals out: $y = A @ x$

@ multiplies and sums; * multiplies and stops

$$\underbrace{[1, 2, 5] * [10, 5, 4]} = [10, 10, 20]$$

*: elementwise, and it STOPS

Matrix product versus elementwise product

@ multiplies and sums; * multiplies and stops

$$\underbrace{[10, 10, 20]}_* \xrightarrow{\text{sum}} \underbrace{40}_{@ \text{ adds this on top}}$$

Matrix product versus elementwise product

@ multiplies and sums; * multiplies and stops

$$\underbrace{[10, 10, 20]}_* \xrightarrow{\text{sum}} \underbrace{40}_{\text{@ adds this on top}}$$

- * is elementwise (same shape out); @ multiplies and sums

Matrix product versus elementwise product

@ multiplies and sums; * multiplies and stops

$$\underbrace{[10, 10, 20]}_* \xrightarrow{\text{sum}} \underbrace{40}_{\text{@ adds this on top}}$$

- * is elementwise (same shape out); @ multiplies and sums
- they are different operations one keystroke apart

Using `*` where you meant `@` fails silently

`(COIN[0]*x).sum()`

`= 40.0`

`=`

`COIN[0] @ x`

`= 40.0`

- on compatible shapes, `*` produces numbers with **no warning**, just the wrong ones

Using `*` where you meant `@` fails silently

`(COIN[0]*x).sum()`

`= 40.0`

`=`

`COIN[0] @ x`

`= 40.0`

`COIN[0] * x`

`= [10, 10, 20], the WRONG thing`

- on compatible shapes, `*` produces numbers with **no warning**, just the wrong ones
- and often the wrong shape, which then poisons everything downstream

Using `*` where you meant `@` fails silently

`(COIN[0]*x).sum()`

`= 40.0`

`=`

`COIN[0] @ x`

`= 40.0`

`COIN[0] * x`

`= [10, 10, 20], the WRONG thing`

- on compatible shapes, `*` produces numbers with **no warning**, just the wrong ones
- and often the wrong shape, which then poisons everything downstream
- the only defense: say the intended output shape out loud before you type

Using `*` where you meant `@` fails silently

`(COIN[0]*x).sum()`

`= 40.0`

`=`

`COIN[0] @ x`

`= 40.0`

`COIN[0] * x`

`= [10, 10, 20], the WRONG thing`

- on compatible shapes, `*` produces numbers with **no warning**, just the wrong ones
- and often the wrong shape, which then poisons everything downstream
- the only defense: say the intended output shape out loud before you type

Using `*` where you meant `@` fails silently

`(COIN[0]*x).sum()`

`= 40.0`

`=`

`COIN[0] @ x`

`= 40.0`

`COIN[0] * x`

`= [10, 10, 20], the WRONG thing`

- on compatible shapes, `*` produces numbers with **no warning**, just the wrong ones
- and often the wrong shape, which then poisons everything downstream
- the only defense: say the intended output shape out loud before you type

`@` and `*` give different shapes from the same inputs



Section 7

Slicing: views versus copies

When does touching a slice quietly touch the original?



■ Slicing: views versus copies

A basic slice shares memory, so writing through it changes X

		sm.0	sm.1	sm.2
X	oats	3	1	2
	p.butter	2	0	2
	whey	2	3	0
	banana	1	2	0

`sm0 = X[:,0]`
a basic slice: a VIEW

- you set smoothie 0's banana to 99 via `sm0`, but column 0 of X changed too

■ Slicing: views versus copies

A basic slice shares memory, so writing through it changes X

		sm.0	sm.1	sm.2
X	oats	3	1	2
	p.butter	2	0	2
	whey	2	3	0
	banana	99	2	0

`sm0 = X[:, 0]`
a basic slice: a VIEW

`sm0[3] = 99`
wrote through to X

- you set smoothie 0's banana to 99 via `sm0`, but column 0 of X changed too
- the slice and X share memory: writing through one changes the other

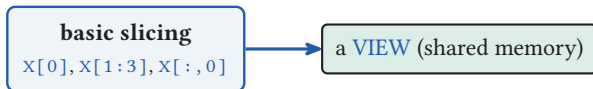
Basic slices are views; index-lists and masks are copies

basic slicing

`x[0], x[1:3], x[:, 0]`

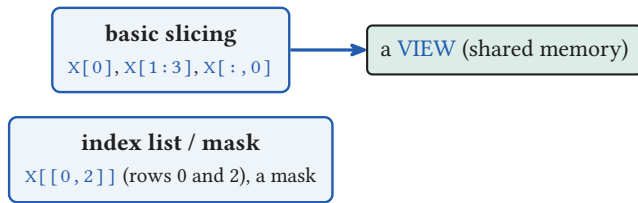
- a colon means “all of this axis”; a view reuses memory through new shape/stride metadata

Basic slices are views; index-lists and masks are copies



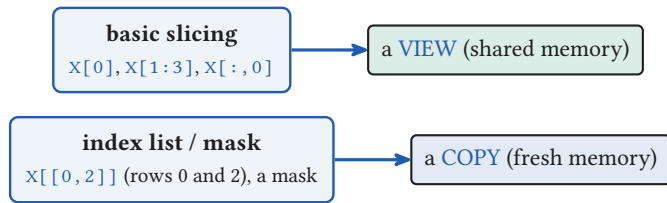
- a colon means “all of this axis”; a view reuses memory through new shape/stride metadata
- an **index list** (`x[ [ 0, 2 ] ]`) or a boolean mask returns a fresh **copy** (NumPy’s “fancy” indexing)

Basic slices are views; index-lists and masks are copies



- a colon means “all of this axis”; a view reuses memory through new shape/stride metadata
- an **index list** (`x[[0, 2]]`) or a boolean mask returns a fresh **copy** (NumPy’s “fancy” indexing)
- writing through a view mutates the original; a copy is independent

Basic slices are views; index-lists and masks are copies



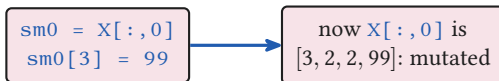
- a colon means “all of this axis”; a view reuses memory through new shape/stride metadata
- an **index list** (`x[[0, 2]]`) or a boolean mask returns a fresh **copy** (NumPy’s “fancy” indexing)
- writing through a view mutates the original; a copy is independent

A deliberate `.copy()` breaks the sharing

```
sm0 = X[:, 0]  
sm0[3] = 99
```

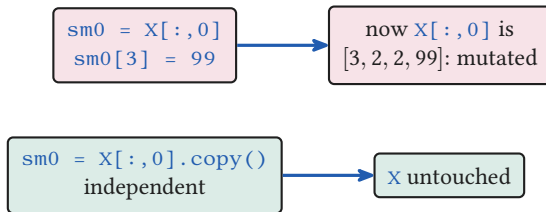
- views are fast and memory-free, but surprise you by changing data you forgot was shared

A deliberate `.copy()` breaks the sharing



- views are fast and memory-free, but surprise you by changing data you forgot was shared
- intend to mutate a slice without disturbing the original? take `.copy()` deliberately

A deliberate `.copy()` breaks the sharing



- views are fast and memory-free, but surprise you by changing data you forgot was shared
- intend to mutate a slice without disturbing the original? take `.copy()` deliberately
- prefer `X[i, j]` over `X[i][j]` (which builds an intermediate view)



Section 8

Masking and axis reductions

How do we count the rows that pass a test, and fold a table to totals?

8

One table, two folds: filter and reduce

a filter

how many smoothies use oats?

- look along the oats row $[3, 1, 2]$, flag a 1 where positive, sum the flags

One table, two folds: filter and reduce

a filter

how many smoothies use oats?

a fold

totals down columns? or across rows?

- look along the oats row [3, 1, 2], flag a 1 where positive, sum the flags
- per-nutrient totals run **across rows**; each smoothie's grand total runs **down a column**

One table, two folds: filter and reduce

a filter

how many smoothies use oats?

a fold

totals down columns? or across rows?

- look along the oats row [3, 1, 2], flag a 1 where positive, sum the flags
- per-nutrient totals run **across rows**; each smoothie's grand total runs **down a column**
- same table, two folds, two completely different meanings

A mask is one flag per smoothie; summing them counts

$$\text{mask} = \left[\mathbb{1}\{x_{0j} > 0\} \right]_j$$

- a 1 where the condition holds, 0 where it fails: one flag per smoothie

A mask is one flag per smoothie; summing them counts

$$\text{mask} = \left[\mathbb{1}\{x_{0j} > 0\} \right]_j$$

- a 1 where the condition holds, 0 where it fails: one flag per smoothie
- summing the indicators **counts** the smoothies that pass

All three smoothies use oats, so the mask sums to 3

		sm.0	sm.1	sm.2
X	oats	3	1	2
	p.butter	2	0	2
	whey	2	3	0
	banana	1	2	0

- a mask is an **indicator**: 1 where true, 0 where false

All three smoothies use oats, so the mask sums to 3

		sm.0	sm.1	sm.2
X	oats	3	1	2
	p.butter	2	0	2
	whey	2	3	0
	banana	1	2	0
	<code>x[0, :] > 0</code>	1	1	1

- a mask is an **indicator**: 1 where true, 0 where false
- `X[:, X[0, :] > 0]` keeps the smoothies (columns) that pass (a copy)

All three smoothies use oats, so the mask sums to 3

		sm.0	sm.1	sm.2
X	oats	3	1	2
	p.butter	2	0	2
	whey	2	3	0
	banana	1	2	0
X[0, :] > 0		1	1	1

sum = 3

- a mask is an **indicator**: 1 where true, 0 where false
- `X[:, X[0, :] > 0]` keeps the smoothies (columns) that pass (a copy)
- summing the mask **counts**: here all three use oats

A reduction folds the array, and the axis you name collapses

a reduction folds the array; the axis you NAME collapses

- on the (3, 3) labels: `axis=0` collapses nutrients, one number per smoothie, shape (3,)

A reduction folds the array, and the axis you name collapses

a reduction folds the array; the axis you NAME collapses

- on the (3, 3) labels: `axis=0` collapses nutrients, one number per smoothie, shape (3,)
- `axis=1` collapses smoothies, one number per nutrient, shape (3,)

A reduction folds the array, and the axis you name collapses

a reduction folds the array; the axis you NAME collapses

- on the (3, 3) labels: `axis=0` collapses nutrients, one number per smoothie, shape (3,)
- `axis=1` collapses smoothies, one number per nutrient, shape (3,)
- `keepdims=True` keeps the axis as size 1: (3, 3) over axis 0 stays (1, 3), so it lines back up

The axis you name is the one that collapses

`sum(axis=0)`

	sm.0	sm.1	sm.2
kcal	339	210	196
protein	26.4	26.3	8.4
fat	13.5	2.8	11.4

`sum(axis=1)`

	sm.0	sm.1	sm.2
kcal	339	210	196
protein	26.4	26.3	8.4
fat	13.5	2.8	11.4

The axis you name is the one that collapses

sum(axis=0)			
	sm.0	sm.1	sm.2
kcal	339	210	196
protein	26.4	26.3	8.4
fat	13.5	2.8	11.4
	▼	▼	▼
	378.9	239.1	215.8
per smoothie			

sum(axis=1)			
	sm.0	sm.1	sm.2
kcal	339	210	196
protein	26.4	26.3	8.4
fat	13.5	2.8	11.4

The axis you name is the one that collapses

`sum(axis=0)`

	sm.0	sm.1	sm.2
kcal	339	210	196
protein	26.4	26.3	8.4
fat	13.5	2.8	11.4

↓ ↓ ↓

378.9	239.1	215.8
-------	-------	-------

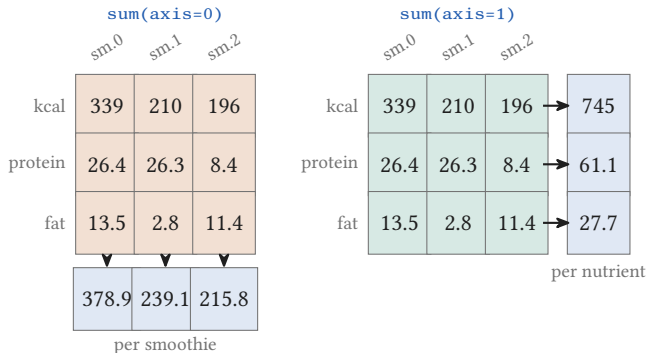
per smoothie

`sum(axis=1)`

	sm.0	sm.1	sm.2	
kcal	339	210	196	→ 745
protein	26.4	26.3	8.4	→ 61.1
fat	13.5	2.8	11.4	→ 27.7

per nutrient

The axis you name is the one that collapses



the named axis collapses; the other one survives



Section 9

Broadcasting

How do arrays of different shapes combine, without writing a loop?

9

Centering subtracts one mean row from every row

center

each label minus the per-nutrient average

- you want `labels - mean` to subtract one average row from every row, no loop

Centering subtracts one mean row from every row

center

each label minus the per-nutrient average

- you want `labels - mean` to subtract one average row from every row, no loop
- but the two arrays have DIFFERENT shapes: `(3, 3)` and `(3, )`

Centering subtracts one mean row from every row

center

each label minus the per-nutrient average

- you want `labels - mean` to subtract one average row from every row, no loop
- but the two arrays have DIFFERENT shapes: `(3, 3)` and `(3, )`
- how do arrays of different shapes combine, element by element?

Right-align the shapes; a size-1 axis stretches

$(3, 3)$ against $(1, 3) \rightarrow (3, 3)$

Right-align the shapes; a size-1 axis stretches

$$(3, 3) \text{ against } (1, 3) \rightarrow (3, 3)$$

- right-align the two shapes; axis by axis: equal, or one is 1 (it stretches, virtually)

Right-align the shapes; a size-1 axis stretches

$(3, 3)$ against $(1, 3) \rightarrow (3, 3)$

- right-align the two shapes; axis by axis: equal, or one is 1 (it stretches, virtually)
- otherwise it is an error, and no data is ever actually copied

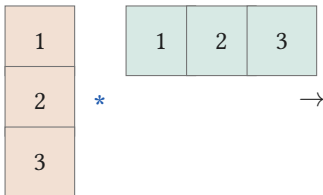
Right-align the shapes; a size-1 axis stretches

$(3, 3)$ against $(1, 3) \rightarrow (3, 3)$

- right-align the two shapes; axis by axis: equal, or one is 1 (it stretches, virtually)
- otherwise it is an error, and no data is ever actually copied
- `None` (a.k.a. `np.newaxis`) inserts a size-1 axis: `v[:, None]` is $(3, 1)$, `v[None, :]` is $(1, 3)$

Broadcasting: a column * a row stretches to a grid

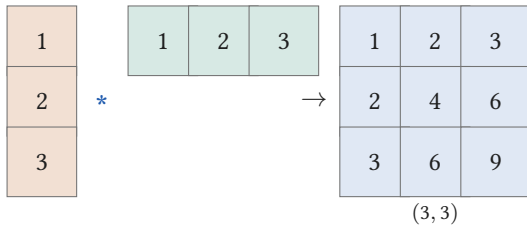
`v[:,None] (3,1)` `v[None,:] (1,3)`



- `labels - labels.mean(axis=0, keepdims=True)` centers each row, no loop

Broadcasting: a column * a row stretches to a grid

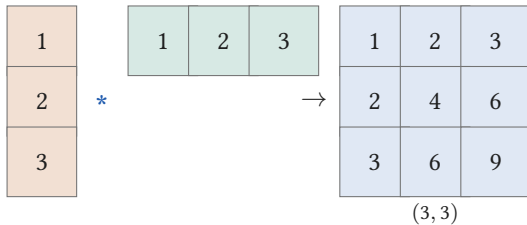
`v[:,None] (3,1)` `v[None,:] (1,3)`



- `labels - labels.mean(axis=0, keepdims=True)` centers each row, no loop
- a column * a row you meant elementwise silently becomes (B, B) , the most common silent bug

Broadcasting: a column * a row stretches to a grid

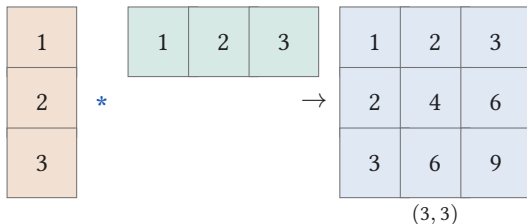
`v[:,None] (3,1)` `v[None,:] (1,3)`



- `labels - labels.mean(axis=0, keepdims=True)` centers each row, no loop
- a column * a row you meant elementwise silently becomes (B, B) , the most common silent bug

Broadcasting: a column * a row stretches to a grid

`v[:,None] (3,1)` `v[None,:] (1,3)`



- `labels - labels.mean(axis=0, keepdims=True)` centers each row, no loop
- a column * a row you meant elementwise silently becomes (B, B) , the most common silent bug

result suddenly two-dimensional? suspect this



Section 10

Tensors and devices

What changes when these same arrays move onto a GPU?

10

A PyTorch tensor mirrors the ndarray, letter for letter

NumPy array X

- `A_t @ X_t` gives the same labels `[[339, 210, 196], ...]`; `.sum(axis=0)` the same per-smoothie totals

A PyTorch tensor mirrors the ndarray, letter for letter



- `A_t @ X_t` gives the same labels `[[339, 210, 196], ...]`; `.sum(axis=0)` the same per-smoothie totals
- exactly one new word appears: the `device`

A PyTorch tensor mirrors the ndarray, letter for letter



same shapes, same @, same labels, letter for letter

- `A_t @ Xt` gives the same labels `[[339, 210, 196], ...]`; `.sum(axis=0)` the same per-smoothie totals
- exactly one new word appears: the `device`

A tensor lives on a device, and every operand must share it

'cpu'
main processor

- a tensor mirrors the ndarray: same construction, shapes, @, .sum(axis=...), .T

A tensor lives on a device, and every operand must share it

'cpu'
main processor

'cuda'
NVIDIA GPU

- a tensor mirrors the ndarray: same construction, shapes, @, .sum(axis=...), .T
- `torch.from_numpy` / `Xt.numpy()` SHARE memory: the view trap now spans both libraries

A tensor lives on a device, and every operand must share it

'cpu'
main processor

'cuda'
NVIDIA GPU

'mps'
Apple-silicon GPU

- a tensor mirrors the ndarray: same construction, shapes, @, .sum(axis=...), .T
- `torch.from_numpy` / `Xt.numpy()` SHARE memory: the view trap now spans both libraries
- iron rule: every operand of an operation must be on the **same** device

A tensor lives on a device, and every operand must share it

'cpu'
main processor

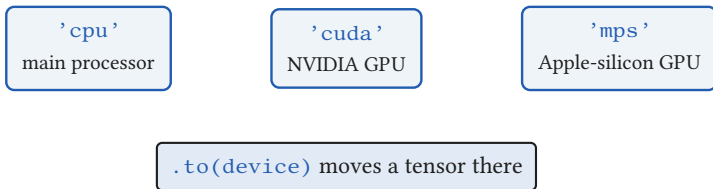
'cuda'
NVIDIA GPU

'mps'
Apple-silicon GPU

`.to(device)` moves a tensor there

- a tensor mirrors the ndarray: same construction, shapes, `@`, `.sum(axis=...)`, `.T`
- `torch.from_numpy` / `Xt.numpy()` SHARE memory: the view trap now spans both libraries
- iron rule: every operand of an operation must be on the **same** device

A tensor lives on a device, and every operand must share it



- a tensor mirrors the ndarray: same construction, shapes, `@`, `.sum(axis=...)`, `.T`
- `torch.from_numpy` / `Xt.numpy()` SHARE memory: the view trap now spans both libraries
- iron rule: every operand of an operation must be on the **same** device

mixing a CPU and a GPU tensor errors; `.to(device)` the straggler

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$
- read (m, n) as “eats an n -list, returns an m -list”

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$
- read (m, n) as “eats an n -list, returns an m -list”
- one product transforms many inputs: $A @ B, A @ X$

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$
- read (m, n) as “eats an n -list, returns an m -list”
- one product transforms many inputs: $A @ B$, $A @ X$
- $@$ multiplies **and sums**; $*$ is elementwise (and broadcasts)

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$
- read (m, n) as “eats an n -list, returns an m -list”
- one product transforms many inputs: $A @ B$, $A @ X$
- $@$ multiplies **and sums**; $*$ is elementwise (and broadcasts)
- the named axis collapses; basic slices are views; masks count; normalize to length 1

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$
- read (m, n) as “eats an n -list, returns an m -list”
- one product transforms many inputs: $A @ B$, $A @ X$
- $@$ multiplies **and sums**; $*$ is elementwise (and broadcasts)
- the named axis collapses; basic slices are views; masks count; normalize to length 1

Summary

- a matrix is a **table of rates**: amounts in, totals out, $y = A x$
- read (m, n) as “eats an n -list, returns an m -list”
- one product transforms many inputs: $A @ B$, $A @ X$
- $@$ multiplies **and sums**; $*$ is elementwise (and broadcasts)
- the named axis collapses; basic slices are views; masks count; normalize to length 1

a matrix is a table of rates; this afternoon, a machine *learns* it